

Keypoint

2025年12月9日 0:16

1.数据的大小端和对齐存储 P62

2.精简指令系统计算机 (RISC) VS CISC

i. RISC: 边界对齐

3.字长: CPU数据通路的宽度

机器字长=通用寄存器位数=CPU内部运算器的位数

存储字长=MDR位数

MAR=主存的地址线位数

指令字长取决于指令的功能和格式: 单字长? 半字长

不同的计算机意味着编译生成的机器指令条数不同

4.cache速度/性能提升:

i. $T_c + (1-p)T_m$

ii. cache比主存快10倍仅表示 $T_m = 10T_c$, 不能表示速度快10倍

5.单位换算

i. ms, us, ns, ps

ii. KB, MB, GB, TB

6.时钟频率含义: 每s多少周期

7.注意进制 (二 or 十六?)

8. $2^{16} = 65536$

9.加法溢出判断

操作数符号相同, 结果符号与操作数不同

OF=1用于有符号数的溢出判断

CF=1, 当作无符号数的溢出判断

10.需要填写计算机组成考虑寄存器

11.对程序员透明含义

12.注意数据表示是从0次方开始

概述

2025年12月9日 11:24

一.冯诺依曼体系架构

- i. Data 和instru以二进制形式同等地位存放在存储器，计算机自动逐条取出指令执行
- ★ ii. 五个基本部件
- iii. 采用存储程序的工作方式
- iv. 指令由操作码和地址码组成

二、计算机特点

- 1.内存层次化
- 2.处理器并行化
- 3.软硬件层次化结构

三、计算机性能

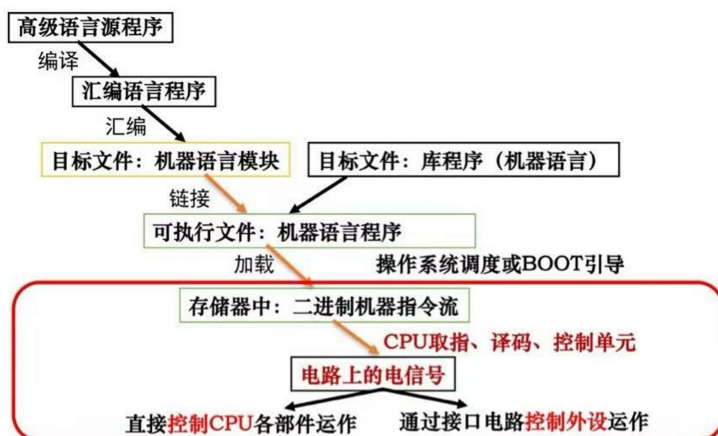
- 1.算法
 - i. 数据结构、算法设计
 - 语句数量
- 2.创建程序并且翻译成机器指令的软件——涉及编译原理
 - 对应机器指令
- 3.执行效率——计算机组成原理，操作系统
 - 指令执行速度，IO执行速度
- 4.存储器系统性能

任何时候单CPU运行一个程序

文本文件&二进制文件

程序=算法+数据

四、程序执行过程



软硬件层次化结构

2025年12月9日 11:36

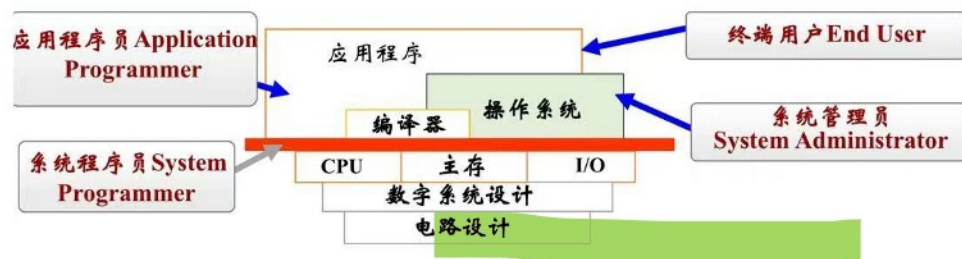
系统软件

- 1.操作系统OS
 - 输入输出
 - 分配存储空间
 - 支持多个程序同时使用计算机
- 2.编译器
- 3.汇编器

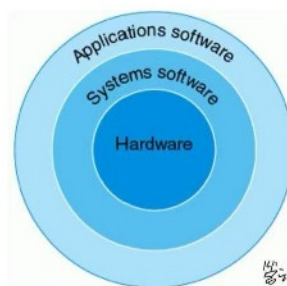
应用软件



Computer Hierarchy (计算机的层次结构)



- 计算机系统层次结构中，指令系统是软/硬件的交界面
- 不同用户工作在不同层次，看到的计算机是不一样的
- 计算机的功能和性能是由机器的指令系统集中体现出来



CPU

2025年12月15日 0:23

组成1

1. control unit

2. datapath

- i. ALU
- ii. R
 - 通用寄存器组
 - PC
 - IR

组成2

1. 运算器

- ALU
- 通用寄存器组
- 程序状态字寄存器

2. 控制器

- PC: $PC+1$, 1指指令的字节数
- IR: 位数=指令字长
- ID: 指令译码器

★ 用户可见寄存器

1. 通用寄存器组 (含基址/变址寄存器)
2. 程序状态字寄存器PSW: ALU运算的状态信息, 条件码
3. 程序计数器PC
4. 基址寄存器

基本指标

2025年12月15日 0:26

★ 一、两个基本指标

- 1.完成单个任务时间：如程序响应时间
- 2.单位时间完成的任务量：如系统吞吐率

二、响应时间

- 1.CPU时间
 - i. 用户CPU时间-->CPU性能
 - ii. 系统CPU时间
- 2.其他时间

三、CPU执行时间

- 1.计算机X比计算机Y快n倍： $T(Y)/T(X)=n$
- 2.CPU time=指令数*CPI/f

四、MIPS&MFLOPS

- 1.MIPS=IC/执行时间*10⁶=f/CPI*10⁶
IPS=f/平均CPI
- 2.MFLOPS

五、阿曼达定律：改进

★ 六、硬件技术指标

- 1.主频：与功耗成正比
- 2.运算速度：MIPS=f/CPI*10⁶、MFLOPS, PFLOPS=每秒10¹⁵次浮点运算
- 3.运算精度
- 4.存储容量
 - i. 主存
 - ii. 辅存

七、其他

IS设计

功耗

2025年12月15日 0:26

一、动态功耗

1. CMOS主要能量功耗
2. 主要是晶体管翻转能量
3. 负载电容 $\times v^2 \times$ 开关频率（与时钟频率有关）

二、静态功耗

1. $v \times$ 漏电流
2. 随晶体管数量增加而增加

计算机发展

2025年12月15日 0:53

一、预测

1. 摩尔：每18个月
2. 安迪和比尔：软件消耗

二、单处理器

1. 处理器性能 = 主频 * IPC
2. 性能提升主要手段：提升工作主频

三、多核处理器

四、并行处理/计算

1. 加速比公式（Amdahl定律）：上限
 - i. 问题规模不改变
 2. G's 定律： $S = N + (1 - N) * s'$
- 可并行化处理

五、大规模并行处理

概述

2025年12月15日 15:44

一、指令系统

1.设计原则

i. 完备性

- 输入输出
- 数据传送
- 算术运算
- 逻辑运算
- 系统控制
- 程序控制：改变PC

ii. 有效性

iii. 规整性

iv. 兼容性

2.主要内容

- 指令格式，指令寻址方式，操作类型以及对应的操作数
- 操作数的类型和寻址方式，存放方式
- 程序可访问的内容
- 指令执行的控制方式

★ 二、两种类型指令系统计算机P202

1.CISC

- i. 各种指令都可以访问存储器
- ii. 有专用寄存器
- iii. 微程序控制
- iv. 难编译优化
- v. 变长指令字

2.RISC

- i. R-R工作
- ii. 通用寄存器，减少访存
- iii. 组合逻辑电路控制
- iv. 优化编译系统
- v. 定长指令字

指令格式

2025年12月15日 15:57

一、指令组成

1. 操作码 (唯一): op code
2. 地址码 (多个)
 - i. 操作数
 - ii. 寻址方式

二、op code

1. 定长
- ★ 2. 变长 (扩展op code)

不允许短码是长码的前缀, 短操作码不能与长操作码的前面部分代码相同

注意: 计算机按字节编址, 则指令位数一定是字节的倍数

三、地址码

1. 零地址: 堆栈 (SP寄存器)
2. 一地址
3. 二地址
4. 三地址 (RISC)
5. 多地址

寻址方式

2025年12月15日 16:57

Op code+目的操作数+Mod+A

- 指令寻址
 - 顺序寻址：PC+1（指令长度，与编址方式，指令字长有关）
- 数据寻址

一、立即数寻址

- 1.操作数本身
- 2.MOV AX, 1000H

二、存储器直接寻址

MOV AX, [1000H]

三、寄存器直接寻址

MOV BX, AX

四、存储器间接寻址

- 1.存储操作数地址，再由其得到操作数
- 2.MOV R1, @(1000H)

五、寄存器间接寻址

MOV AX, [BX]

六、偏移寻址

- 1.直接寻址 (R) + 寄存器间接寻址（寻到存储器）
- 2.EA=(R)+A（注意编址方式）
- 3.分类
 - i. 相对寻址：PC
 - ii. 基址寻址
 - 基址寄存器由操作系统确定
 - 适用于多道程序设计，为程序或数据分配存储空间
 - iii. 变址寻址：线性表元素存取
 - 变址寄存器内容由用户决定

七、堆栈寻址

POP Rn

八、复合寻址

九、寻址方式确定

- 1.与op code绑定
- 2.专门寻址方式位

十、单/双操作数

MIPS指令系统

2025年12月23日 8:41

★ 一、32位通用寄存器

1. 注意操作数一定是在R 或者imm，M里面需要使用lw/sw配合访问
2. \$sp注意预留堆栈和释放堆栈（自上向下）：sp-4（预留）

表 1-1 MIPS32 中通用寄存器的约定用法

寄存器名字	约定命名	用途
\$0	zero	总是为 0
\$1	at	留作汇编器生成一些合成指令
\$2、\$3	v0、v1	用来存放子程序返回值
\$4~\$7	a0~a3	调用子程序时，使用这 4 个寄存器传输前 4 个非浮点参数
\$8~\$15	t0~t7	临时寄存器，子程序使用时可以不用存储和恢复
\$16~\$23	s0~s7	子程序寄存器变量，改变这些寄存器值的子程序必须存储旧的值并在退出前恢复，对调用程序来说值不变
\$24、\$25	t8、t9	临时寄存器，子程序使用时可以不用存储和恢复
\$26、\$27	\$k0、\$k1	由异常处理程序使用
\$28 或 \$gp	gp	全局指针
\$29 或 \$sp	sp	堆栈指针
\$30 或 \$fp	s8/fp	子程序可以用来做堆栈帧指针
\$31	ra	存放子程序返回地址

二、指令格式

注意：有无u（符号），有无v（针对位移，variable）

1. R-type

- i. 运算
- ii. 比较
- iii. 逻辑：nor, xor
- iv. 移位：注意算术移位和逻辑移位sll&sllv

2. I-type（与imm有关）

- i. 16位imm
- ii. 算术：add
- iii. 比较
- iv. 逻辑
- v. lw/sw

3. J-type

- i. 26bit imm
- ii. PC高四位+26imm+00

三、寻址方式

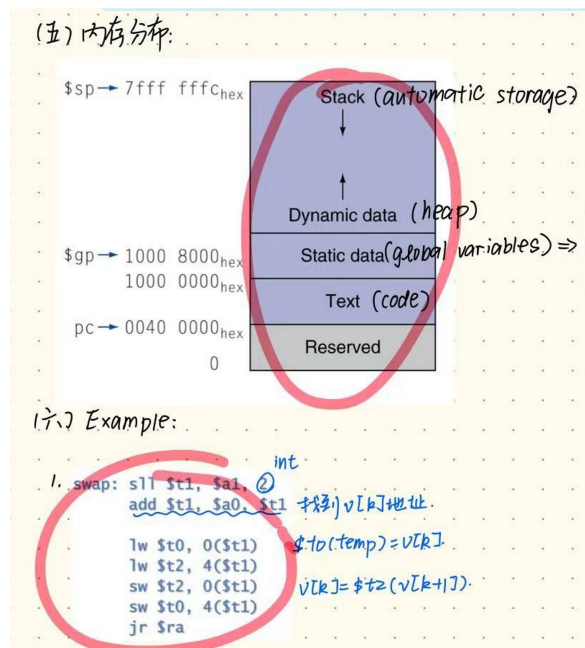
Op code决定

- i. R-type
- ii. I-type
- iii. J-type：偏移量有符号整数

四、数据存储

边界对齐，大端

变量在M中



五、指令类型

1. 算术逻辑运算

- load大数: lui&ori
- 乘法: hi&lo&mfhi&mflo (mf为R-type)
- 除法: lo (商), hi (余数)
- 比较
- 逻辑
 - 位操作 (xori看做无符号)
 - 取反: xor
 - 置零
 - 置1
 - 取位
 - 移位 (shift)
 - sll逻辑移位
 - sra算数右移

2. 数据传输

- sw/lw
- imm为offset可正可负
- data和variables存储在R
- lb/sb注意存储方式 (大小端)

3. 条件分支: $pc = pc + 4 + 4 * offset$

4.无条件跳转

J label	J-type	Op=2
Jar label	J-type	Op=3
Jr \$t0	R-type	Func=8
Jalr \$t0,\$ra	R-type	Func=9,存在\$ra

六、指令应用

1.

```
lui $5, 0x5678
ori $5, $5, 0xABCD
```

2.

① $x: 32\text{bit} \times 32\text{bit} = 64\text{-bit}$

$hi:lo = \$rs * \rt

□ upper 32 bits go into **hi**, lower 32 bits go into **lo**

!To access result, use two new instructions

□ **mfhi**: move from hi
mfhi rd
□ move the 32-bit half result from hi to \$rd

□ **mflo**: move from lo
mflo rd
□ move the 32-bit half result from lo to \$rd

For example:
mult \$9,\$8
mflo \$9
mfhi \$8

② $\div: \text{div } rs, rt.$

$lo = \$rs / \$rt \Rightarrow \text{mfhi}$

$hi = \$rs \% \$rt. \Rightarrow \text{mflo.}$

3.

数据

2025年12月15日 22:48

一、表示

- 1.非规格化计算公式，解码为0或者255时表示的特殊的数
- 2.浮点加法P99：对阶，注意精度丢失
- 3.ASCII码正负以及十进制表示P76
- 4.8421码正负
- 5.IEEE754浮点表示P73

二、度量

1.KB	2^{10}	10^3
2.MB	2^{20}	10^6
3.GB	2^{30}	10^9
4.TB	2^{40}	10^{12}

三、存储

- 1.编址方式
- 2.连续对齐
- ★ 3.大端存储&小端存储

四、其他概念

- 1.字长=datapath宽度=CPU内部总线宽度=运算器位数=通用寄存器宽度
- 2.数据表示：能被计算机硬件直接识别的数据类型

概述（重要）

2025年12月5日 10:57

1. 地址译码器

i. 单译码：

- 地址线12根 p对应4096个状态，需要4096根译码线

ii. 双译码

- 地址线12根 pX、Y方向各6根， $64*64=4096$ 个状态，128根译码线

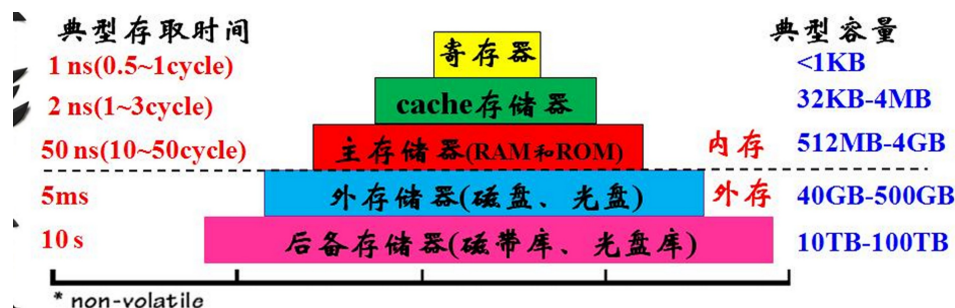
2. 片选与读写控制逻辑

- | | | | | |
|----------|--|------|--|------|
| CS*, CE* | | OE*读 | | WE*写 |
|----------|--|------|--|------|

★ 3. 芯片引脚安排

- 地址线：芯片字数
- 数据线：芯片位数
- 控制线：WE* CS*
- 电源线：1
- 接地线：1

★ 4. 层次式存储器结构



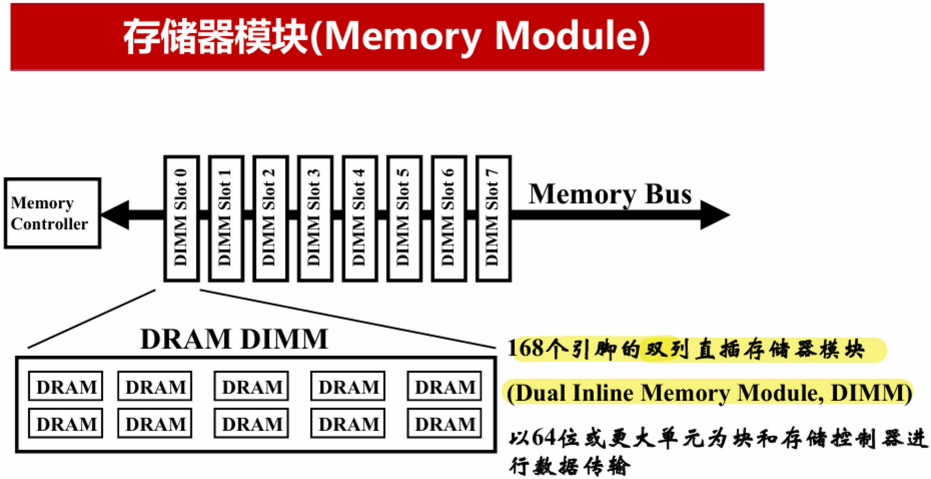
5. 存储器按存取方式分类P81

- RAM
- ROM
- 串行访问存储器：按物理地址的先后顺序寻址

- 顺序存取存储器（如磁带）
- 直接存取存储器（如磁盘、光盘）

一、RAM

1. 存储器模块：



每次访存操作总是在某一个内存条内进行！

- 主存储器由若干内存条（存储模块）组成
- DIMM
- 主流DDR4（双倍数据速率）

2. DRAM

i. 芯片结构

- 两次输入地址分别为行地址和列地址（地址线增加一条，增加两位地址，四倍容量）
- 分时复用。
- **RAS* & CAS***
- 数据线采用两根：**Din 和 Dout**
- 先行地址再列地址
- “仅行地址有效”刷新：间隔一定时间

3. DRAM & SRAM

都易失

SRAM和DRAM存储器的特性

	DRAM	SRAM
存储原理	电容	触发器 (双稳态电路)
集成度	高 (较简单)	低
芯片引脚	少 (分时复用)	多
速度	慢(10X)	快(1X)
价格	低(1X)	高(100X)
刷新	有	无
应用	主存、帧缓冲区	高速缓存存储器

DRAM：地址复用，需按行定时刷新，行缓冲器（SRAM）

二、ROM

1.介绍：

- i. 非破坏性读出，无需再生
- ii. 随机存取（random）
- iii. 非易失

2.分类

- i. 固定掩膜MROM：
 - 只能读，不能修改
 - 结构简单，集成度高，价格便宜
- ii. 一次改写的PROM（可编程ROM）
 - 熔断丝型PROM为主流
 - PROM既按地址又按位寻址，编程时是按位编程且须脱机编程
- iii. 可多次改写ROM
 - A. EPROM，可擦除编程
 - 不能在线擦除和编程
 - 常用：块擦除电可编程的快擦写存储器Flash Memory
 - 不对单个存储单元擦除，仅以片为单位擦除
 - 集成度高，耐久性好
 - B. EEPROM
 - 可在线擦除和编程，可单地址擦除和编程
 - 芯片封装容易
 - 保持时间长
 - 写慢，集成度低
 - C. 闪存：常见SSD固态硬盘即flash芯片组成，MOs

主存储器逻辑设计

2025年12月6日 0:14

1. 存储器构成

- (半导体) 存储芯片: 由存储芯片 ($mK \times n$ 位/片) 构成存储器 ($MK \times N$ 位)
- 其它逻辑芯片: 例如地址寄存器、地址译码器、刷新控制电路等

2. 设计基本原则:

i. 使用要求

- 存储器性质: 主存? cache? $\leftrightarrow$ DRAM/SRAM/ROM
- 指标
- 环境条件

ii. 实际条件

- 功能要求

3. 设计

Step one 系统设计

确定存储结构组成、外围电路、芯片类型

- 存储芯片类型: SRAM、DRAM、ROM

- 外围芯片类型: ECL、TTL、CMOS等

- 选址方法: 重合法、线选法

- 系统结构: 多体、单体, 编址方式

i. 确定类型:

- ROM $\rightarrow$ 系统程序、标准子程序
- RAM $\rightarrow$ 用户程序

ii. 确定数量: ROM和RAM地址范围

Step two 逻辑设计

- ★ 容量拓展
- ★ 负载计算与分析
 - 速度估算
 - 纠错

步骤

- i. 确定芯片数量
- ii. 确定芯片连接
- iii. 负载计算和分配
 - 注意RAS并行驱动，每行都要有驱动
- iv. 速度估算

Step three工艺设计

存储容量扩展

2025年12月6日 0:25

注:

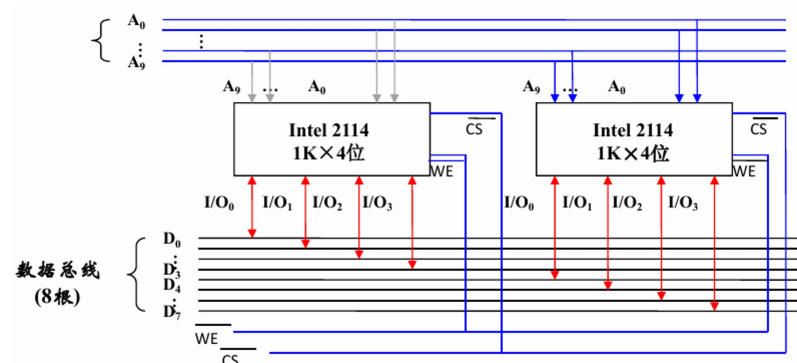
①区分存储器（存储单元）地址码和位数与芯片的地址码和位数

②注意芯片引脚安排

一、位扩展：存储芯片($m_k \times n$ 位/片)构成存储器($m_k \times N$ 位)

1.需存储芯片数： N/n （天花板）片

2.



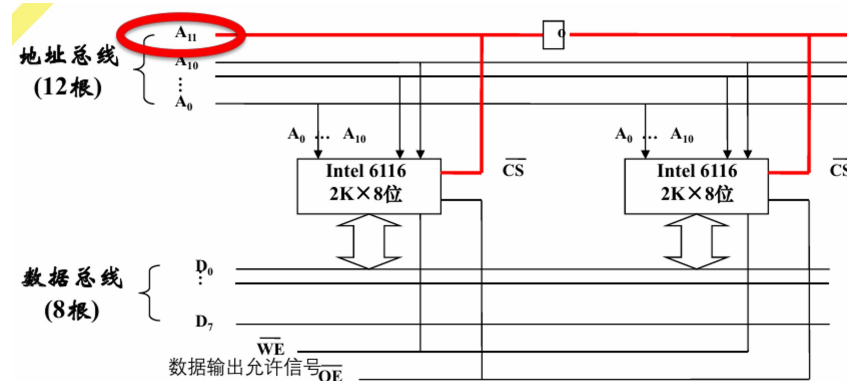
二、字扩展

1.存储芯片($m_k \times n$ 位/片)构成存储器($M_k \times n$ 位)

2.需存储芯片数： M/m （天花板）

3.-CS（片选）：单独引出，与增加的高位地址码(存储器地址码位数减芯片的地址码位数)的译码结果连接

4.

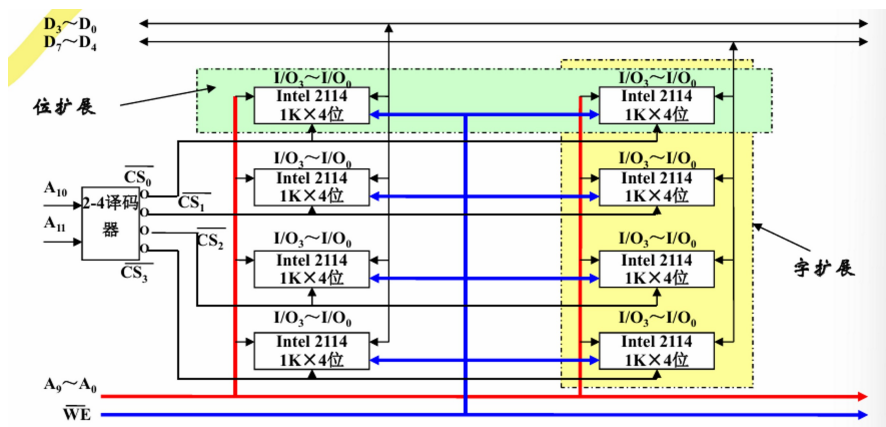


三、字、位同时扩展

1.存储芯片($m_k \times n$ 位/片)构成存储器($M_k \times N$ 位)

2.需存储芯片数： $\square M/m \square \square N/n \square$ 片（天花板）

3.



负载分析与速度计算

2025年12月6日 10:00

一、驱动与负载概述：

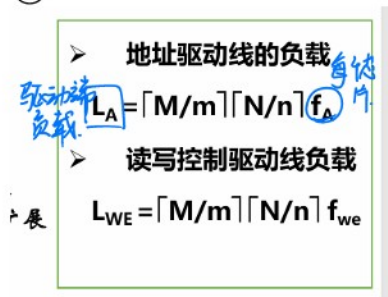
1. 存储器的逻辑设计中，外围电路芯片是驱动，存储芯片的各端点就是负载

2. 双极型芯片各端点为电流负载；MOS型芯片各端点 为电容负载

- 双极结型晶体管 (BJT) 核心元件，依靠电子和空穴载流子导电
- MOS金属氧化物半导体，仅电子或者空穴

★ 3. 负载因数 f 端点名：存储芯片某种端点中的一个端点的 负载量称为该种端点的负载因数（驱动端负载和芯片负载）

①



② 外围逻辑电路需要驱动的负载端加重，以致超出其驱动能力

二、速度估计

1. 速度延迟

外围电路传输要引起时延，使存储器的系统存储周期、存取时间比存储芯片的存储周期、存取时间要长

2. 速度估算应比要求的快10%

3.

系统存储周期 T_{SM}

- $T_{SM} = T_M + t_D + t_R$
- T_M 为芯片存储周期：查芯片数据手册
- t_D 为系统传输时延：由外围电路逻辑级数确定
- t_R 为系统恢复时间：通过系统测试获得

存储器与CPU连接

2025年12月9日 19:21

一、连接方式

1. 地址线连接（地址总线）
2. 数据线的连接（数据总线）
3. 控制线的连接（读写）
 - i. $\overline{MREQ}$ 0 → 主存 1 → 输入输出端口

二、通信方式

1. 异步
2. 同步

★ 三、解题

1. 分析对应二进制地址码、确定总容量
2. 确定芯片类型和数量
3. 分配地址线
 - i. 低位地址码、使能端
 - ii. 片选信号
 - iii. 片选（如三八译码器）使能端

高速缓冲存储器cache

2025年12月9日 19:44

程序访问局部性原理

2025年12月9日 19:44

1. 存储层次化结构有效的原因
2. 分类（指令和数据分别讨论）：
 - i. 时间局部性：for循环
 - ii. 空间局部性：顺序执行指令，数组

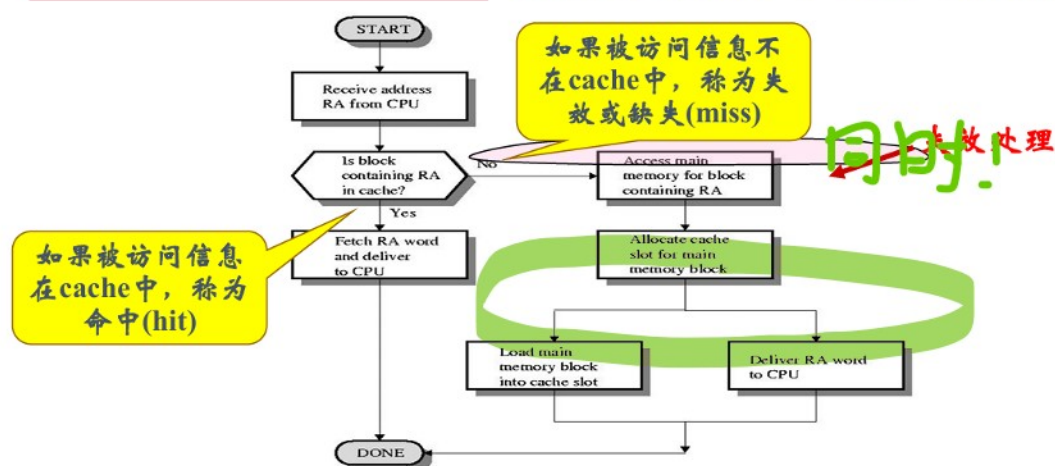
Cache

2025年12月9日 19:56

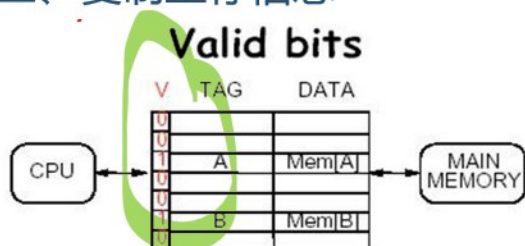
一、结构

- 1.组成: SRAM
- 2.v: 直接制作在CPU芯片内, 速度几乎与CPU一样快
- 3.一般将Cache和主存的存储空间都划分为若干大小相同的块 (主存中称为: 块 Block、Cache中称为: 行line/slot)
- 4.存放最近访问的数据 (主存副本)
- 5.tag+标志位 (必有1有效位) +数据块

二、操作过程



三、复制主存信息



四、访问时间

- 1.平均访问时间

□命中 (Hit) : 要访问的信息在Cache中

- Hit Rate(命中率 p): 在Cache中的概率
- Hit Time (命中时间 T_c): 访问Cache所需时间, 包括: 判断时间 + Cache访问

□失效 (Miss) : 要访问的信息不在Cache中

- Miss Rate (缺失率/失效率 $1 - p$) = $1 - (\text{Hit Rate})$
- Miss Penalty (失效损失 T_m): 从主存将一块数据读取到Cache所需时间, 包括访问主存块, 向上逐层传输块直至将数据块放入发生缺失的那一层所需时间。

命中时间 << 失效损失

□平均访问时间 = $p \times T_c + (1-p) \times (T_m + T_c) = T_c + (1-p) \times T_m$

提高平均访问速度, 必须提高命中率!



2. 存储访问时间

则存储系统的等效访问周期 T 为:

$$T = T_c \times H + T_m \times (1 - H)$$

3. 加速比

$$S_p = \frac{T_m}{T}$$

五、多级cache系统设计

1. 单级/多级

2. 分立/联合

- L1 cache命中时间比hit重要, 减少hit获得较短时钟周期, 防止结构冒险
- L2 hit比t更重要, 降低损失率以减少访问主存缺失损失

cache和主存映射

2025年12月9日 20:16

一、把主存划分成大小相等的主存块(Block)

二、Cache中存放一个主存块的对应单位称为行(line) 或槽(Slot)或项 (Entry)或块(Block)

★ 三、映射（见笔记本）

1.求数据cache总容量：tag+valid+行长/数据块大小（注意B和bit的换算）

2.由内存地址定位cache（以直接映射为例）：主存地址-->主存块-->cache块-->mod cache行

3.注意区分cache直接映射mod与组相联组数的求取

cache组成

- Tag
- Valid（可能还有其他标记项）
- 数据块

直接映射	cache行号=主存块号mod cache行数	tag+cache行+块内地址	只能映射到固定某一行
全相联映射	与cache每行tag进行对比	tag+块内地址	可以映射到cache所有行
组相联映射	cache组号=主存块号 mod cache组数	tag+cache组号+块内地址	N路组相联可以映射到N行

四、MISS RATE

1.影响因素：

- cache大小
- Block 块大小（数据交换以cache块为单位）
- cache映射方式（看容量）

2.失效类型

- 强制失效
- 容量失效
- 冲突失效

五、cache一致性

1.cache内容更新，主存不改变

2.I/O设备直接读写内存

3.多个CPU都有各自私有的cache且共享主存

4.cache写机制

i. 写命中

- Write through
 - 同步更新
 - CPI增加（主存访问时间慢）
 - 优化：cache 和MM之间buffer
- Write back
 - 仅写cache
 - Dirty bit：修改位为1，且从cache里替换才写入主存

ii. 写不命中：空间局部性，读一个块的开销

- 写分配
 - 更新主存，再主存块装入cache
- 写不分配
 - 直接写主存

六、cache替换

1.FIFO

标记最先进入

2.LRU

- i. 空间局部性
- ii. 计数器（LEU位）：注意计数器变化规则
- iii. 和FIFO都可能抖动

3.random 替换

七、cache设计

1.分离cache结构（一般L1）

- 指令cache
- 数据cache

2.多级cache

虚拟存储器

2025年12月20日 9:15

一、存储器管理

1. 单道程序

2. 多道程序

os系统动态执行划分任务

二、虚拟存储器基本概念

1. 将内外存统一存储管理（缓存管理机制，不是物理存储器）
2. 用户程序存储空间分页，**按需调页**（部分disk，部分MM）
3. 由主存和辅存共同构成
4. 对应用程序员来说透明，对系统程序员不透明

三、分区

1. 简单分区

2. 可变长分区

存储器碎片

四、分页

1. 程序块（页，大）-->存储块（页框，不用连续）
2. os系统为每个进程生成一个页表
3. 页表：VA（程序指令所用地址）-->PA（存放指令或数据实际内存地址）
 - i. 页表在主存中
 - ii. 逻辑地址转换靠硬件MMU

五、组织方式

硬件与os system共同协作

注意偏移量（拼接）和寻找页地址（页+段内页号，是移动）

1. 段式管理

- i. 程序模块化：函数 or 功能-->逻辑独立性，但碎片
 - ii. 段表：主存中可再定位段
 - 段名
 - 装入位：缺段
 - 段长：地址出界
-

- 起始地址（实际存储）：与段内地址（offset）拼接-->主存实地址
 - 访问方式：保护违例
- iii. 分段对程序员不透明，可见，分页对程序员透明
- iv. 段表基址寄存器（多用户虚地址）
- 段表长度
 - 段表基地址

★ 2.页式虚拟存储器

- i. VA和MA统一大小划分成页
- 虚页：虚页号+页内地址
 - 实页：实页号+页内地址
- ii. **页表**：程序员可见逻辑地址-->处理器使用PA
- MM中
 - Os system为每个程序建立一个页表
 - 进程大小-->页表项数
- iii. **页表基址寄存器**
- 每个进程都有一个页表基址寄存器，存放该进程的页表首地址
- iv. CPU执行指令先将逻辑地址-->PA（MMU）
- v. 缺页代价大：
- 软件处理：缺页处理os
 - Write back
 - 全相联映射
 - random替换
- vi. 异常
- 缺页

Disk-->MM，考虑替换（dirty-bit），指令阻塞

- 保护违例

R, R/W, X（只执行），指令阻塞

TLB（快表）

1.why?

无TLB通过页表来会访问两次存储器

2.what?

- i. 依赖页表访问局部性
- ii. cache存储页表项
- iii. SRAM组成，全相联or组相联
- iv. TLB表项=页表表项内容+TLB标记
- 全相联标记为虚拟页号
 - 组相联标记为虚拟页号高位部分，虚拟页号低位部分是TLB组号

3.组织：虚页号+PA+else

- i. 全相联：虚页号为tag

- ii. 组相联：虚页号为tag+index (TLB标记对应高位虚拟页号，而虚拟页号低位作为TLB组号)

4.3miss

- i. TLB：硬件or软件
 - ii. 页表：os软件处理
 - iii. cache (PA)：硬件处理
-

3.段页式虚拟存储器

- i. 程序按模块分段，段内分页（段长为页长的整数倍），MM以页为基本单位
- ii. $\text{Logi A} = \text{段地址（可能基号+段号）} + \text{页地址} + \text{页内偏移量}$

多周期处理器P458

2025年12月31日 15:26

指令类型	取指（同）	译码（同）	执行	访存	写回（注意写回数据和信号的稳定）
R-type	1.存储器读取指令	1.控制逻辑延迟后控制信号更新	ALU运算，输出结果		写寄存器 注：RegDst稳定，RegWr稳定，ALU输出稳定
branch	2.指令保存IR寄存器（具有写使能）	2.寄存器读取指令	1.sub，判断0信号 2.指令执行完成 3.PC更新		
Ori	3.ALU：PC+4	3.译码	1.立即数逻辑右移 2.ALU运算输出output		写回R
Lw	4.保存PC+4到PC输入端（有“写使能”），指令结束时改变	4.ALU：“投机计算”分支地址	计算读地址	从存储器取数	写R
Sw			计算写地址	访存写	

流水线处理器

2025年12月31日 16:19

一、作用

- 1.提升CPU指令吞吐率
- 2.提升工作主频
可通过增加流水线级数来提升，但流水线级数不可过多
- 3.提高处理器的并行性
 - 时间
 - 空间：超标量处理机

二、datapath

- 1.每个周期完成一条指令，一条指令进入流水线，CPI=1
- 2.寄存器读口和写口可看作不同部件
- 3.P512

三、流水线冒险P107

1.结构竞争/资源冲突

- 寄存器访问冲突
 - 读口写口独立
- 访存冲突
 - 单独设置数据存储器 and 指令存储器

2.数据关联竞争：写与取（RAW）

- 硬件：流水线阻塞stall
- 软件：nop空指令（编译器）
- 转发/旁路
 - 能够解决相隔一条的两个运算类指令
- 寄存器写口和读口分别控制在前、后半时钟周期

特殊：载入冒险——lw后紧跟R-type，无法通过转发完全解决

解决方式：

- stall/nop：冻结、清除、禁止PC
- 编译器调整指令顺序

3.控制竞争

- i. Jump
 - ii. Branch
- 判断提前
 - EX阶段判断分支-->STALL2
 - ID阶段-->STALL1
 - 分支预测

- 延迟分支：编译器静态调度
 - Nop
 - stall

4.异常/中断控制冒险

四、要求

- 1.指令长度尽量一致
- 2.指令格式尽量规整
- 3.仅load/store指令访存
- 4.data和instru按边界对齐方式存放

五、性能指标P260

- 1.吞吐率：TP
- 2.加速比：S

六、高级流水线技术

- 1.超标量流水线技术：多发射，同时处理多条指令
 - $CPI < 1$
- 2.超流水线技术：提高流水线主频（流水线功能划分越多，T越短）

异常和中断

2026年1月7日 15:39

一、异常

1.响应

- CPU执行过程中检测到异常/发现中断请求
- 能解决-->回到第i条/i+1条
- 不能解决-->终止

注：具体处理过程由os完成

2.分类

- 软件中断
 - 故障：指令启动后，执行结束前
 - 自陷
- 硬件中断
 - 终止：硬件故障
 - 外中断

二、中断

1.定义：CPU外部

2.分类

- 可屏蔽中断
- 不可屏蔽中断

三、异常中断响应过程

1.关中断

2.保存断点和程序状态

3.识别异常和中断并转到相应的处理程序

数据通路

2026年1月7日 15:01

一、组成

1.组合逻辑电路

- 三态门

2.时序逻辑电路

二、基本结构

1.CPU内部单总线方式

- 暂存
- 三态门
- 一个时钟内只允许一次操作

2.CPU内部多总线方式

3.专用数据通路方式

单周期CPU

2026年1月7日 19:37

- 1.注意保持写数据稳定（控制信号）